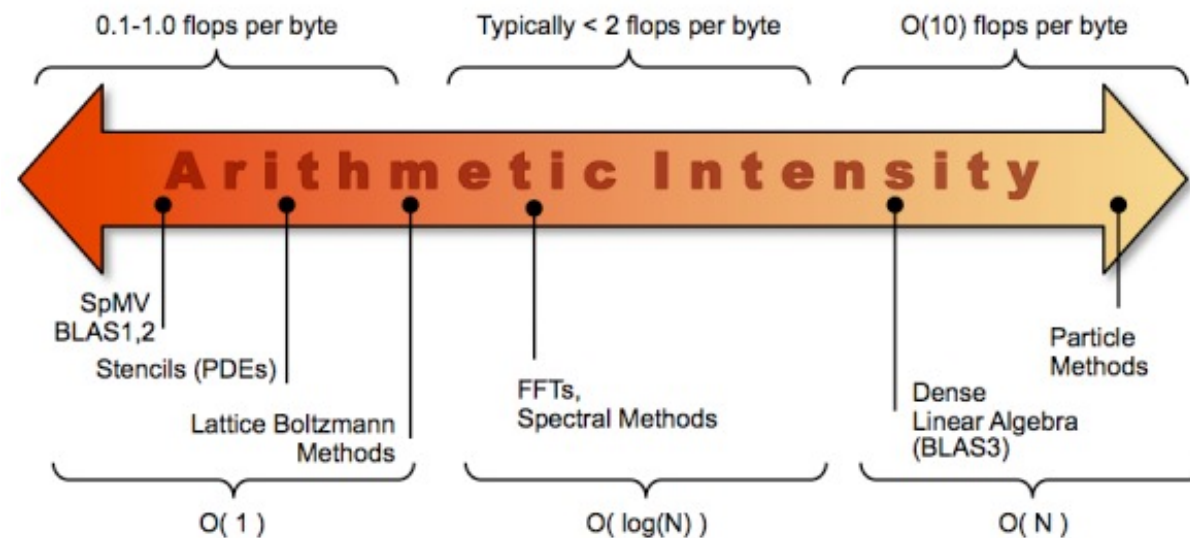


Speculative parallelism

Parallel Algorithms

João Luís Sobral (jls@di ...)

April/2026



Data storage

Regular : vector, matrix

Irregular **pointer based**:
linked-list, tree, graphs

Data accesses

- Regular / **irregular**

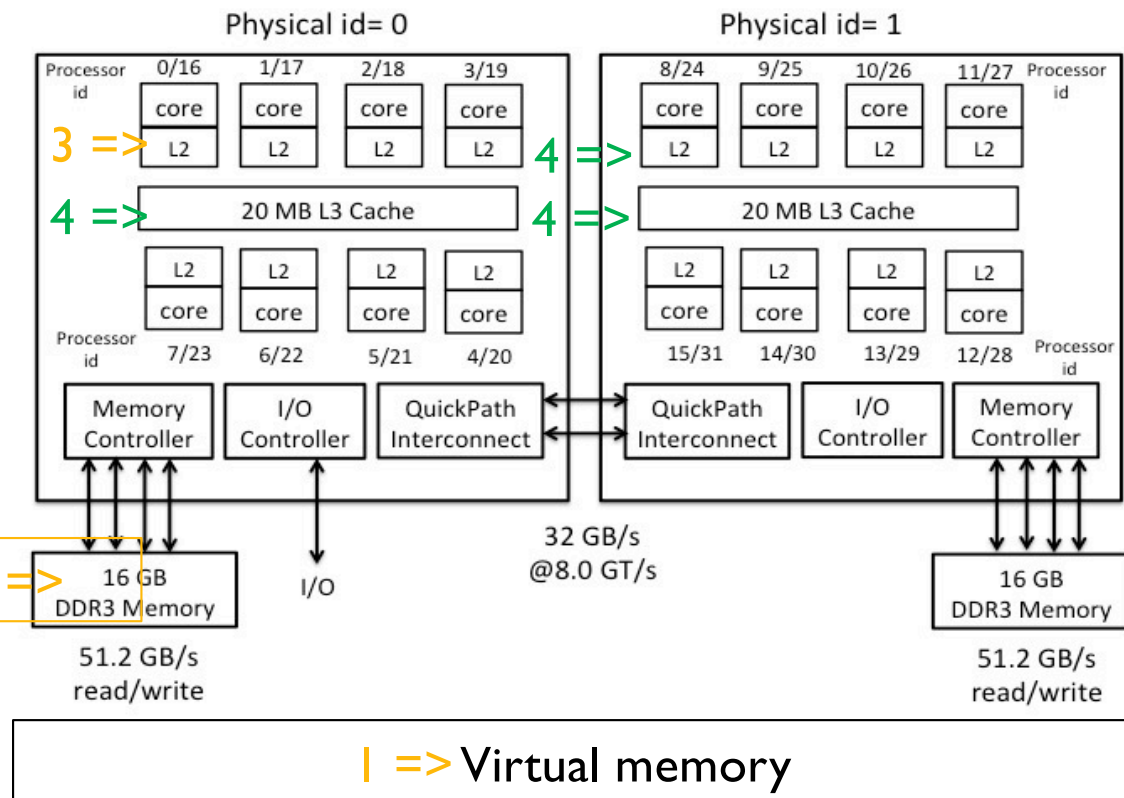
Recap: memory hierarchy (a detailed view)

1 => Data is allocated in **virtual memory**

2 => on Linux: physical memory page(s) are reserved on a memory bank on the “first-touch”

3 => Data is cached on local L3/L2/L1 as threads read the data from the shared memory

4 => Multiple data copies are kept coherent by the hardware



- **Data locality:** the hardware will try to keep the data closer to the PUs (i.e., threads) using it

- Data movement costs?

Recap: memory coherence & consistency

- OpenMP memory consistency model & atomics

- OpenMP uses a **relaxed-consistency shared-memory model**, where threads can maintain their own "temporary view" of memory that may not be immediately consistent with global memory.
- Every **atomic** has **implicit flushes**. By default, these are "strong flushes" on the variable being accessed

- Efficient barrier implementation (source: ChatGPT)

- “Simple barrier implementation using OpenMP atomics”

```
int __attribute__((aligned(256))) count = 0;
int __attribute__((aligned(256))) sense = 0;
```



Memory coherence: avoid false sharing between count and sense (cache line of 256 bytes on A64FX)

```
void omp_atomic_barrier(int nthreads) {
    int my_sense;
```

```
    // Flip local sense
    #pragma omp atomic read
    my_sense = sense; // can be in local view
    my_sense = !my_sense;
```



Memory consistency: ensure a “fresh” value of the shared sense

```
    // Increment counter
    int c;
    #pragma omp atomic capture
    c = ++count;
```



Memory consistency: update a “fresh” value of the shared count (note: implies two flushes)

```
    if (c == nthreads) {
        // Last thread resets and releases others
        count = 0;
        #pragma omp atomic write
        sense = my_sense;
    } else {
        // Spin
        int s;
        do {
            #pragma omp atomic read
            s = sense;
        } while (s != my_sense);
    }
}
```



Memory consistency: ensure memory update of sense
Question: atomic update on count is required?



Memory coherence: spin on local sense (cached value - only cache misses if sense is updated)

Memory consistency: ensure a “fresh” value of sense

Recap: Lock-free Algorithms

Blocking algorithms/data structures

- A blocking algorithm allows one thread to prevent other threads from completing operations on a shared data structure indefinitely
- Example:
 - Thread 0 takes a lock on a node in our linked list
 - Thread 0 is swapped out by the OS, or crashes, or is just really slow (takes a page fault), etc.
 - Now, no other threads can complete operations on the data structure (although thread 0 is not actively making progress modifying it)
- An algorithm that uses locks is blocking regardless of whether the lock implementation uses spinning or pre-emption

Lock-free algorithms

- Non-blocking algorithms are lock-free if some thread is guaranteed to make progress (“systemwide progress”)
 - In lock-free case, it is not possible to preempt one of the threads at an inopportune time and prevent progress by rest of system
 - Note: this definition does not prevent starvation of any one thread

Recap: OpenMP Synchronization

(updated with the **real code** for Parallel Algorithms)

- Solving data races with critical VS atomic

```
5 int main(){
6     double result={0};
7
8     #pragma omp parallel for shared(result)
9     for(int i=0; i<1000000;i++) {
10         result+=sin(i);
11     }
12     printf("%f",result);
13 }
```

Data race

```
...
vmovsd (%r12), %xmm1
.L4: vxorpd %xmm2, %xmm2, %xmm2
vmovsd %xmm1, 8(%rsp)
vcvttsi2sdl %ebx, %xmm2, %xmm0 ; convert to double
call sin
vmovsd 8(%rsp), %xmm1 ; load result into %xmm1
...
vaddsd %xmm0, %xmm1, %xmm1 ; add
...
jne .L4
vmovsd %xmm1, (%r12) ; update result
...
```

%r12 contains a pointer to a shared memory position

local copy!

- Critical:** coarse-grain and pessimist

```
call GOMP_critical_start
vaddsd (%r12), %xmm0, %xmm1 ; load result & add into %xmm1
vmovsd %xmm1, (%r12) ; update result
call GOMP_critical_end
```

Mutual exclusion: ensures that only one thread executes region (e.g., blocks the calling thread if another thread is still executing the region)

- Atomic:** fine-grain and optimistic

```
L4: vaddsd %xmm0, %xmm1, %xmm1
```

```
...
lock cmpxchgq %xmm1, (%r12) ; atomic comp & exchange
// implicit cmpq %rax, (%r12)
jne .L4 ; repeat if update failed
```

Repeat until successful update

This is already a form of speculation
What strategy is the best?

The future of lock-free Algorithms

- Current ARMv8.1-A ISA

atomic instructions can perform simple arithmetic or logical operations on the specified memory location, and return the original value to the processor:

- Compare and swap instructions, CAS and CASP
- Atomic memory operation instructions, LD<OP> and ST<OP>, where OP is one of arithmetic and bitwise operations
- Swap instruction, SWP

- ARMv9.4-A, FEAT_LSE128 extends FEAT_LSE to support atomics on 128-bit data:

- LDCLRP, atomic bit clear on quadword in memory, atomically loads a 128-bit quadword from memory
- LDSETP, atomic bit set on quadword in memory, atomically loads a 128-bit quadword from memory
- SWPP, swap quadword in memory, atomically loads a 128-bit quadword from memory, and stores a new value to memory

Recap: Lock instructions

x86 cmpxchg

- Compare and exchange (atomic when used with lock prefix)

`lock cmpxchg dst, src`



lock prefix (designates operation is atomic)



often a memory address

if (dst ==  EAX)
 ZF = 1 
 dst = src
else
 ZF = 0
 EAX = dst

x86 register

flag register holds result of check

Recap: Lock instructions

Test-and-test-and-set lock

```
void Lock(int* lock) {  
    while (1) {  
        while (*lock != 0);           // while another processor has the lock...  
                                     // (assume *lock is NOT register allocated)  
  
        if (test_and_set(*lock) == 0) // when lock is released, try to acquire it  
            return;  
    }  
}  
  
void Unlock(int* lock) {  
    *lock = 0;  
}
```

Transactional Memory

Raising level of abstraction for synchronization

Previous topic: machine-level atomic operations

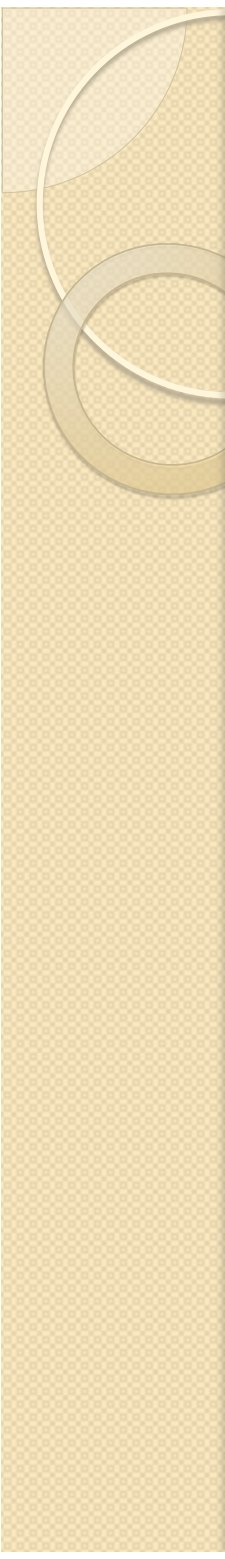
- Test-and-set, fetch-and-op, compare-and-swap, load linked-store conditional

Then we used these atomic operations to construct higher level synchronization primitives in software:

- Locks, barriers
- Lock-free data structures
- We've seen how it can be challenging to produce correct programs using these primitives (easy to create bugs that violate atomicity, create deadlock, etc.)

Today: raising level of abstraction for synchronization even further

- Idea: transactional memory



Transactional Memory

Between a Lock and a Hard Place

Locks force trade-off between

- Degree of concurrency $\Rightarrow$ performance
- Chance of races, deadlock $\Rightarrow$ correctness

Coarse grain locking

- low concurrency, higher chance of correctness
 - E.g. single lock for the whole data structure or all shared memory

Fine grain locking

- high concurrency, lower chance of correctness
 - E.g. hand-over-hand locking

Is there a better synchronization abstraction?

Transactional Memory

Review: ensuring atomicity via locks

```
void deposit(Acct account, int amount)
{
    lock(account.lock);
    int tmp = bank.get(account);
    tmp += amount;
    bank.put(account, tmp);
    unlock(account.lock);
}
```

Deposit is a read-modify-write operation: want “deposit” to be atomic with respect to other bank operations on this account

Locks are one mechanism to synchronize threads to ensure atomicity of update (via ensuring mutual exclusion on the account)

Transactional Memory

Programming with transactions

```
void deposit(Acct account, int amount)
{
    lock(account.lock);
    int tmp = bank.get(account);
    tmp += amount;
    bank.put(account, tmp);
    unlock(account.lock);
}
```



```
void deposit(Acct account, int amount)
{
    atomic {
        int tmp = bank.get(account);
        tmp += amount;
        bank.put(account, tmp);
    }
}
```

Atomic construct is declarative

- Programmer states what to do (maintain atomicity of this code), not how to do it
- No explicit use or management of locks

System implements synchronization as necessary to ensure atomicity

- System could implement `atomic { }` using locks (see this later)
- Implementation discussed today uses optimistic concurrency: maintain serialization only in situations of true contention (R-W or W-W conflicts)

Transactional Memory

Transactional Memory (TM) Semantics

Memory transaction

- An atomic and isolated sequence of memory accesses
- Inspired by database transactions

Atomicity (all or nothing)

- Upon transaction commit, all memory writes in transaction take effect at once
- On transaction abort, none of the writes appear to take effect (as if transaction never happened)

Isolation

- No other processor can observe writes before transaction commits

Serializability

- Transactions appear to commit in a single serial order
- But the exact order of commits is not guaranteed by semantics of transaction

Transactional Memory

Transactional Memory (TM)

In other words... many of the properties we maintained for a single address in a coherent memory system, we'd like to maintain for sets of reads and writes in a transaction.

Transaction:

Reads: X, Y, Z

Writes: A, X

← These memory transactions will either all be observed by other processors, or none of them will. (the effectively all happen at the same time)

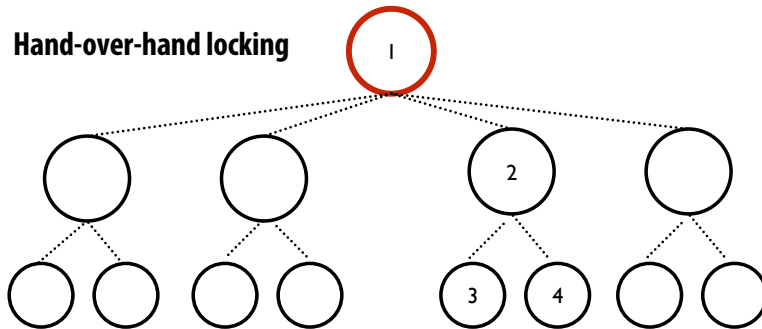
What is the consistency model for TM?

Transactional Memory

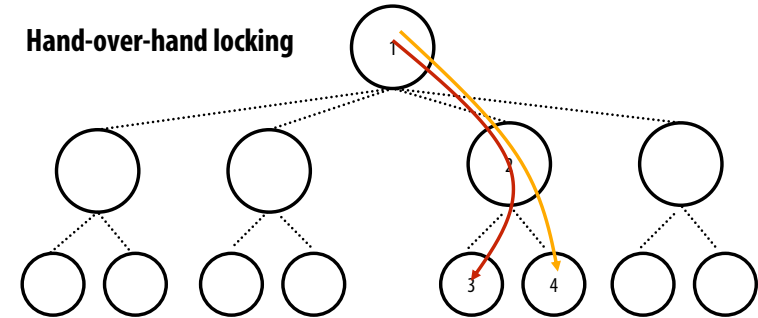
Fine-grained locking example

Goal: modify nodes 3 and 4 in a thread-safe way

Hand-over-hand locking



Hand-over-hand locking

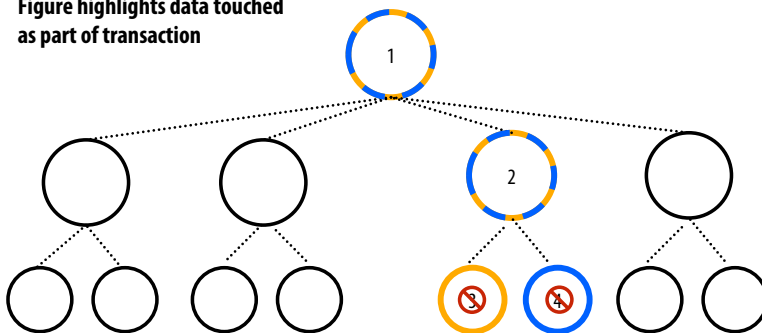


Locking can prevent concurrency

(here: locks on node 1 and 2 during update to node 3 could delay update to 4)

Transactions example

Figure highlights data touched as part of transaction



Transaction A
READ: 1, 2, 3
WRITE: 3

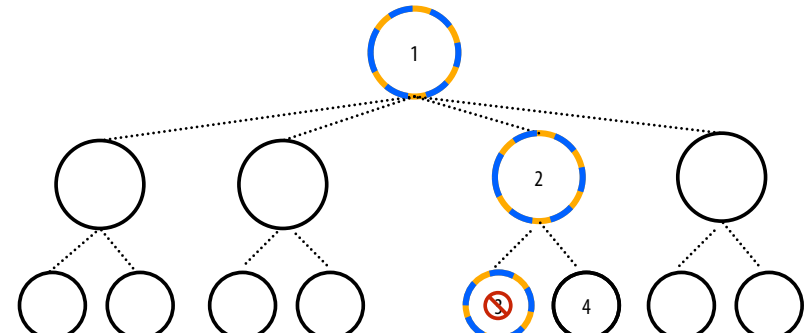
Transaction B
READ: 1, 2, 4
WRITE: 4

**NO READ-WRITE or
WRITE-WRITE conflicts!**
(no transaction writes to data that is
accessed by other transactions)

Slide credit: Austen McDonald

Transactions example #2

(Both transactions modify node 3)



Transaction A
READ: 1, 2, 3
WRITE: 3

Transaction B
READ: 1, 2, 3
WRITE: 3

**Conflicts exist: transactions
must be serialized**
(both transactions write to node 3)

Transactional Memory

Hardware transactional memory (HTM)

Data versioning is implemented in caches

- Cache the write buffer or the undo log
- Add new cache line metadata to track transaction read set and write set

Conflict detection through cache coherence protocol

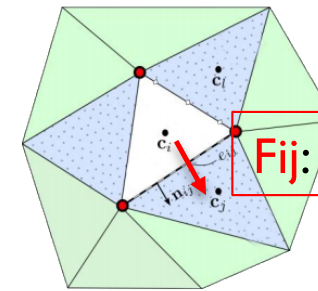
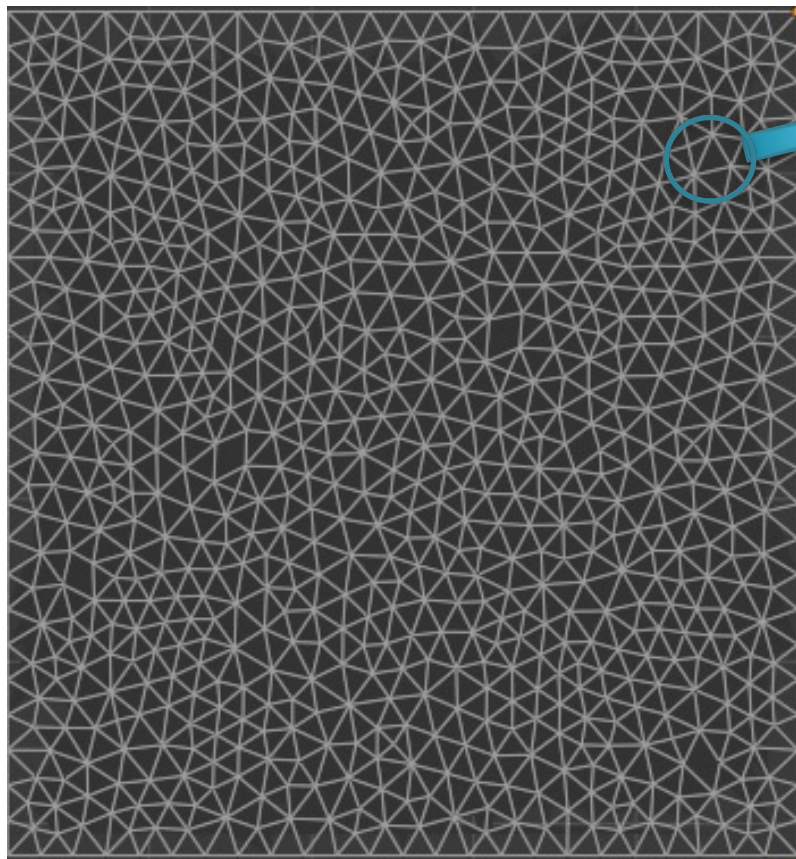
- Coherence lookups detect conflicts between transactions
- Works with snooping and directory coherence

Note:

- Register checkpoint must also be taken at transaction begin (to restore execution context state on abort)

Recap: Sparse matrix – Graph equivalence

- Finite Volume example



Fij: flux across edge e_{ij}

Flux balance for cell c

$$G_c = \sum F_{ij} = \underbrace{[V \cdot n_{ij}]^+}_{\text{velocidade normal}} \phi_i + \underbrace{[V \cdot n_{ij}]^-}_{\text{velocidade normal}} \phi_j - \underbrace{k \frac{\phi_j - \phi_i}{|b_i \cdot b_j|}}_{\text{Difusão}}$$

```
//makeFlux(Mesh2D &m, Vect<double> &phi, Vect<double> &G)
for(int i; i<m.totalEdges(); i++){
    Edge& edge = m.edge[i];
    double nv = edge.nv; // normal_velocity: nij;
    double leftPhi = phi[ edge.left_cell_id ]; // ci
    double rightPhi = phi[ edge.right_cell_id ]; // cj

    // compute the convection contribution
    double Fij = nv * ((nv<0) ? rightPhi : leftPhi);

    // compute the diffusive contribution
    Fij -= edge.k_bibj * (rightPhi-leftPhi);

    G[ edge.left_cell_id ] += Fij;
    G[ edge.right_cell_id ] -= Fij;
}
```

What is the Graph data structure?

Sparse matrix – Graph equivalence

- FVLib example (cont.)

```
1 for N in totalCells do
2   makeFlux(..., F, ...) {
3     for I in totalEdges do
4       if (!FRONTIER[I])
5         leftPhi = PHI[LEFT[I]];
6         rightPhi = PHI[RIGHT[I]];
```

COO (Coordinate)
sparse matrix format



Each edge is a
point in the
sparse matrix

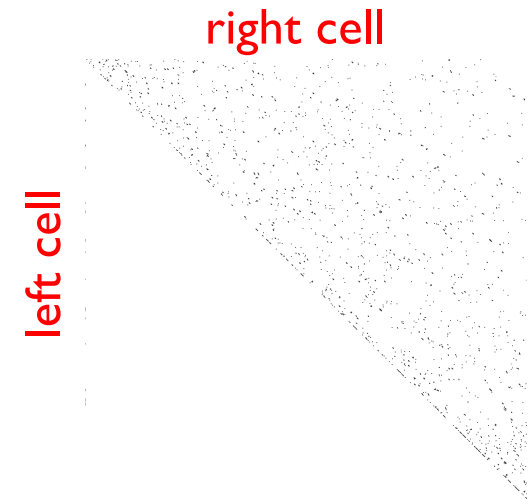


Figura 18.: Matriz original.

- Graph – Sparse matrix equivalence**

- Reducing sparse matrix bandwidth
by renumbering cells

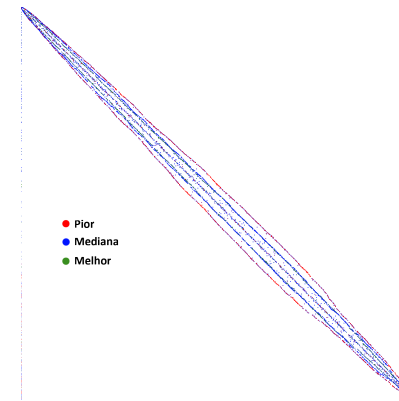
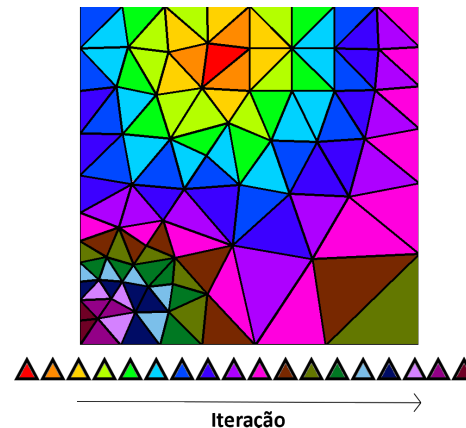
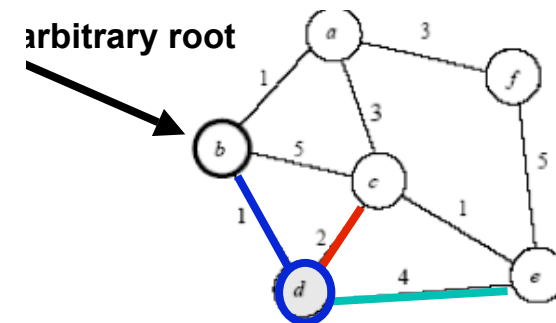


Figura 23.: Matriz de adjacência depois da renumeração.

Recap: Graph Algorithms

- Minimal Spanning Tree: Prim ($O(V \log E)$)
 - touch every vertex in the graph through the smallest weighted edges.

```
CurrentVertex = Random Start Vertex;  
Neighbours    = Neighbours Of(Current Vertex);  
Add To PQ(Neighbours);  
while(Unvisited Vertices Exist) {  
    // lazy removal of previously visited neighbours  
    do {  
        Min Neighbour = Get Minimum(PQ);  
        Current Vertex = Vertex Of(MinNeighbour);  
    } while (is Visited(Current Vertex));  
    visit(Current Vertex);  
    Add To MST(Min Neighbour);  
    Neighbours = Neighbours Of(Current Vertex);  
    // add unvisited neighbours to PQ  
    for each neighbour in Neighbours {  
        if(is Not Visited(Vertex Of(neighbour)))  
            Add To PQ(neighbour);  
    }  
}
```



$O(\log_2 E)$ with a PQ



works by successively adding graph neighbour connections to the PQ and removing the smallest weight connection and marking the touched vertices as visited



$O(\log_2 E)$ with a PQ

Graph Algorithms

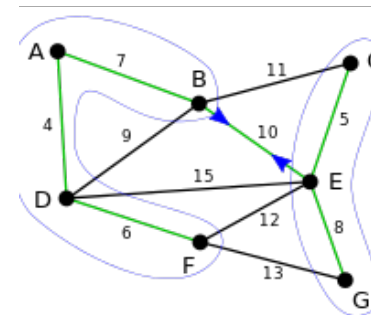
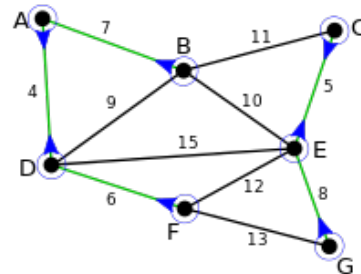
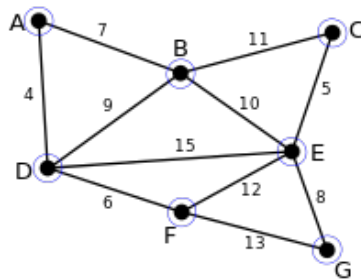
- Minimal Spanning Tree: Boruvka

- The algorithm begins by first examining each vertex and adding the cheapest edge from that vertex to another in the graph, without regard to already added edges, and continues joining these groupings in a like manner until a tree spanning all vertices is completed.

```

0  Input: A connected graph G whose edges have distinct weights
1  Initialize a forest T to be a set of one-vertex trees, one for each vertex of the graph.
2  While T has more than one component:
3      For each component C of T:
4          Begin with an empty set of edges S
5          For each vertex v in C:
6              Find the cheapest edge from v to a vertex outside of C, and add it to S
7          Add the cheapest edge in S to T
8      Combine trees connected by edges to form bigger components
9  Output: T is the minimum spanning tree of G.

```

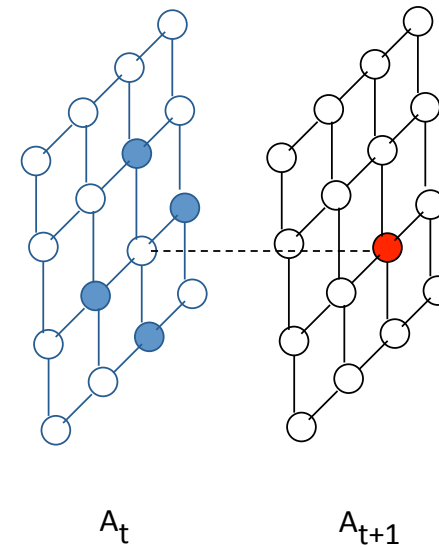


Galois system

- Optimistic (speculative) parallelization framework designed specifically for irregular applications, such as graph algorithms
 - Unlike regular applications where loops can be easily parallelized, irregular applications have complex, data-dependent, or unpredictable memory accesses
- **Motivating Example: Spanning Tree**
 - **Sequential approach:** Pick a node, add neighbors, repeat
 - **Galois Parallel approach:**
 - Pick a set of edges.
 - Use foreach to iterate over edges.
 - If a neighbor hasn't been visited, try to add it.
 - If a conflict occurs (another thread added it), the system handles the rollback.

HPC example

- Finite-difference computation
- Algorithm
 - Operator: five-point stencil
 - Different schedules have different locality
- Regular application
 - Application can be parallelized at compile-time



Jacobi iteration, 5-point stencil

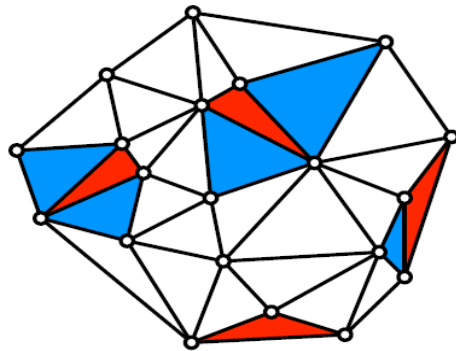
```
//Jacobi iteration with 5-point stencil
//initialize array A
for time = 1, nsteps
  for <i,j> in [2,n-1]x[2,n-1]
    temp(i,j)=0.25*(A(i-1,j)+A(i+1,j)+A(i,j-1)+A(i,j+1))
  for <i,j> in [2,n-1]x[2,n-1]:
    A(i,j) = temp(i,j)
```

Irregular example

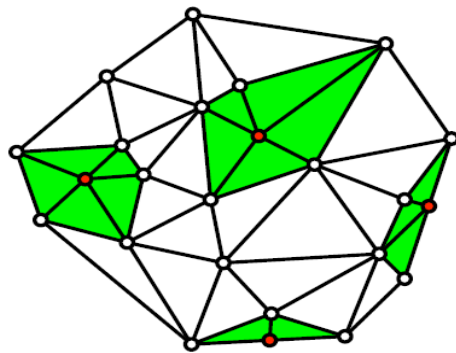
```
Mesh m = /* read in mesh */
WorkList wl;
wl.add(m.badTriangles());
while (true) {
    if (wl.empty()) break;
    Element e = wl.get();
    if (e no longer in mesh)
        continue;
    Cavity c = new
        Cavity();
    c.expand();
    c.retriangulate();
    m.update(c); //update mesh
    wl.add(c.badTriangles());
}
```

- Where is parallelism in program?
 - Loop: do static analysis to find dependence graph
- Static analysis fails to find parallelism.
 - May be there is no parallelism in program?

Data-centric view of algorithm



Before



After

Delaunay mesh refinement (DMR)

Red Triangle: badly shaped triangle

Blue triangles: cavity of bad triangle

- Algorithm
 - composition of unitary **actions** on data structures
- Actions: **operator**
 - DMR: {find cavity, retriangulate, update mesh}
- Composition of actions:
 - specified by a **schedule**
- Parallelism
 - disjoint actions can be performed in parallel
- Parallel data structures
 - graph
 - worklist of bad triangles